Pandas Real-Life Problem

E-Commerce Sales Analysis

Scenario: You Are a Data Analyst at ShopEasy

ShopEasy is an Indian e-commerce company. They have a sales dataset with customer orders, product details, prices, and delivery info. Your job: load the data, clean it, explore it, filter key insights, and prepare it for the business team.

Dataset — Download Link

Dataset	Source	Direct Download URL
E-Commerce Sales Dat	Kaggle (public)	https://raw.githubusercontent.com/dsrscientist/dataset1
Backup / Alternative	GitHub Sample	https://raw.githubusercontent.com/dsrscientist/dataset1

```
How to load: df = pd.read_csv('https://raw.githubusercontent.com/dsrscientist/dataset1/master/e-commerce.csv')
```

Dataset Columns Reference

Column	Description
order_id	Unique order ID
customer_id	Customer identifier
gender	Customer gender (M/F)
age	Customer age (may have missing values)
city	Customer city
product_category	Category of product ordered
product	Product name
quantity	Number of items ordered
price_per_unit	Unit price in INR
total_price	Total order value
payment_method	Cash / Card / UPI
delivery_status	Delivered / Pending / Cancelled

Session 1 — Introduction: Series & DataFrame

Task 1: Create the DataFrame [Session 1]

Load the dataset using pd.read_csv() from the URL above. Store it as df. Then print its type, first 5 rows, and shape.

```
import pandas as pd url =
'https://raw.githubusercontent.com/dsrscientist/dataset1/master/e-commerce.csv' #
Write your code below
```

Write your code here:

Task 2: Explore as a Series [Session 1]

Extract the 'product_category' column as a Pandas Series. Print the Series and its type.

```
# Write your code below (use df from Task 1)
```

Write your code here:

Task 3: Build a Mini DataFrame Manually [Session 1]

Without loading the CSV, manually create a small DataFrame with 3 rows and columns: ['order_id', 'product', 'total_price']. Use your own sample values.

```
import pandas as pd # Write your code below
```

Expected: a 3x3 DataFrame printed to screen

Write your code here:

Session 2 — Reading & Writing Data

Task 4: Read & Inspect [Session 2]

Load the dataset. Use head(10) to see first 10 rows, tail(5) for last 5, and info() to see dtypes and null counts.

```
import pandas as pd url = '...' df = pd.read_csv(url) # Write your code below
```

Write your code here:

Task 5: Summary Statistics [Session 2]

Use describe() to get summary stats for all numeric columns. Find: what is the average total_price? What is the max price_per_unit?

```
# Write your code below
```

Write your code here:

Task 6: Export Cleaned Snapshot [Session 2]

Save the first 100 rows of df to a new CSV file called 'shopeasy_sample.csv' (without the index).

```
# Write your code below
```

Write your code here:

Session 3 — Selecting & Filtering (loc & iloc)

Task 7: Select Specific Columns [Session 3]

Using loc, select only the columns ['customer_id', 'product', 'total_price'] for the first 10 rows.

```
# Write your code below #
```

Write your code here:

Task 8: Select by Position (iloc) [Session 3]

Using iloc, select rows 5 to 15 and columns 0 to 3 (positional).

```
# Write your code below
```

Write your code here:

Task 9: Filter High-Value Orders [Session 3]

Filter all orders where total_price > 10000. How many such orders exist? Print the count and the filtered DataFrame.

```
# Write your code below
```

Write your code here:

Task 10: Multi-Condition Filter [Session 3]

Filter orders where: - product_category == 'Electronics' AND - delivery_status == 'Delivered' AND - total_price > 5000 Print the result.

```
# Write your code below
```

Write your code here:

Task 11: Query Method [Session 3]

Use df.query() to find all orders from customers in 'Mumbai' with payment_method == 'UPI'.

```
# Write your code below
```

Write your code here:

Session 4 — Sorting & Reindexing

Task 12: Sort by Price Descending [Session 4]

Sort the full DataFrame by total_price in descending order. Print the top 10 most expensive orders.

```
# Write your code below
```

Write your code here:

Task 13: Sort by Multiple Columns [Session 4]

Sort by product_category (A→Z) first, then by total_price (highest first) within each category.

```
# Write your code below
```

Write your code here:

Task 14: Reset Index After Filter [Session 4]

Filter all 'Cancelled' orders. Then reset the index so it starts from 0. Print the first 5 rows of the result.

```
# Write your code below #
```

Write your code here:

Task 15: Reindex with Custom Order [Session 4]

After sorting by total_price descending, reindex the result to have a clean 1-based index (1, 2, 3...). Hint: use reset_index then add 1.

```
# Write your code below
```

Write your code here:

Session 5 — Handling Missing Values

Task 16: Detect Missing Values [Session 5]

Check every column for missing values. Print: - Total nulls per column - Percentage of nulls per column

```
# Write your code below #
```

Write your code here:

Task 17: Drop Rows with Missing Data [Session 5]

Drop any rows where 'age' or 'total_price' is missing. Print shape before and after.

```
# Write your code below
```

Write your code here:

Task 18: Fill Missing Age with Median [Session 5]

Fill missing values in the 'age' column with the median age. Verify: after filling, df['age'].isnull().sum() should be 0.

```
# Write your code below
```

Write your code here:

Task 19: Replace with Category Mode [Session 5]

Fill missing values in 'payment_method' with the most frequent (mode) payment method.

```
# Write your code below
```

Write your code here:

Task 20: Full Clean Pipeline [Session 5]

Write a complete cleaning pipeline in sequence: Step 1: Drop rows where order_id is missing Step 2: Fill missing age with median Step 3: Fill missing payment_method with mode Step 4: Drop any remaining rows with nulls Step 5: Print final shape and isnull().sum()

```
# Write your code below — all 5 steps in order
```

Write your code here:

Bonus Challenge — Combine All Sessions

Boss Task: Using all skills from Sessions 1–5, write a single script that:

1. Loads the dataset (Session 2)
2. Cleans all missing values — age→median, payment→mode, drop rest (Session 5)
3. Filters only 'Delivered' orders with total_price > 3000 (Session 3)
4. Sorts the result by total_price descending and resets index (Session 4)
5. Selects columns: customer_id, city, product_category, total_price (Session 3)
6. Exports the final result to 'vip_orders.csv' (Session 2)

Write your code here: